

Universidad Tecnológica Nacional

Facultad Regional Tucumán

Ingeniería en Sistemas de Información

Individual | *Junio de 2026*



Trabajo Práctico Final

Implementación de Blog Personal sobre Contenedores LXC en Proxmox



Estudiante:

Lucas Dante Depetris
Legajo N°52432

Docente:

Cátedra de Virtualización

Materia:

Virtualización

Índice

1. Introducción	3
2. Contexto	3
3. Objetivos	3
3.1. Objetivo General	3
3.2. Objetivos Específicos	3
4. Marco Teórico	4
4.1. Virtualización y Contenedores LXC	4
4.2. Proxmox VE	4
4.3. Arquitectura Web	4
4.4. Proxy Inverso	4
5. Desarrollo del Proyecto	4
5.1. Planificación	4
5.2. Infraestructura Implementada	5
5.3. Pila Tecnológica	5
5.4. Backend	5
5.5. API	6
5.6. Modelo de Datos	7
5.7. Frontend	8
5.8. Administración del Blog	8
5.9. Monitoreo e Información Técnica	9
5.10. Configuración de nginx	9
5.11. Publicación del Informe	9
5.12. Publicación bajo un Prefijo de URL	9
6. Pruebas Realizadas	10
7. Resultado Final	10
8. Conclusión	11

1. Introducción

El presente informe documenta el Trabajo Práctico Final de la materia Virtualización, cuyo objetivo fue implementar un servicio de blog personal sobre infraestructura de contenedores en Proxmox. La solución desarrollada integra una aplicación web con frontend, backend, base de datos y servidor web, distribuida en dos contenedores LXC.

El sistema permite publicar y administrar entradas de blog, mostrar información personal del alumno, cargar una imagen de perfil, consultar información técnica de la infraestructura y ofrecer el informe final en formato PDF desde el propio sitio. El desarrollo se realizó con React, FastAPI, PostgreSQL y nginx, respetando la separación de responsabilidades entre aplicación y base de datos.

La implementación se desplegó en la red provista para la práctica, utilizando los contenedores 43362480A para la aplicación y 43362480DB para la base de datos. El acceso externo previsto se realiza a través del dominio `nap.frt.utn.edu.ar`.

2. Contexto

La virtualización permite ejecutar múltiples entornos aislados sobre una misma infraestructura física. En este trabajo se utilizó Proxmox VE como plataforma de administración de contenedores LXC, lo que permitió crear servicios independientes con recursos asignados de CPU, memoria, disco y red.

El caso de uso elegido fue un blog personal. Aunque se trata de una aplicación simple desde el punto de vista funcional, resulta adecuada para aplicar conceptos centrales de la materia: creación de contenedores, direccionamiento IP, instalación de servicios, exposición mediante proxy inverso, comunicación entre contenedores y despliegue de una aplicación web completa.

3. Objetivos

3.1. Objetivo General

Implementar un blog personal funcional sobre contenedores LXC administrados con Proxmox, separando la aplicación web de la base de datos y publicando el servicio mediante nginx.

3.2. Objetivos Específicos

- Crear y configurar dos contenedores LXC para la aplicación y la base de datos.
- Instalar y configurar PostgreSQL como motor de persistencia.
- Desarrollar un backend con FastAPI para exponer la API del blog.
- Desarrollar un frontend con React y TypeScript para la interfaz de usuario.
- Configurar nginx como servidor web y proxy inverso hacia el backend.
- Implementar autenticación para las operaciones administrativas.
- Permitir la administración de publicaciones, perfil personal, foto y registro de cambios.
- Publicar el informe del trabajo práctico en formato PDF desde el sitio.

4. Marco Teórico

4.1. Virtualización y Contenedores LXC

La virtualización consiste en abstraer recursos físicos para ejecutar sistemas aislados sobre un mismo host. A diferencia de una máquina virtual completa, un contenedor LXC comparte el kernel del sistema anfitrión, pero mantiene su propio espacio de procesos, red, sistema de archivos y configuración.

Esta característica hace que los contenedores sean livianos y adecuados para servicios pequeños. En el trabajo se asignaron recursos limitados a cada contenedor: 128 MB de RAM, 1 CPU y 8 GB de disco, lo que exigió una implementación simple y eficiente.

4.2. Proxmox VE

Proxmox VE es una plataforma de virtualización que permite administrar máquinas virtuales y contenedores desde una interfaz web. En este proyecto se utilizó para crear los contenedores, asignar recursos, configurar la red y operar los servicios necesarios.

Cada contenedor recibió un nombre asociado al DNI del alumno y una dirección IP dentro de la red 172.16.90.0/24. La IP se definió siguiendo el criterio de la consigna: 172.16.90.ID, donde ID corresponde al identificador asignado por Proxmox.

4.3. Arquitectura Web

La aplicación se organizó en tres capas principales:

- **Frontend:** interfaz de usuario construida con React, TypeScript y Vite.
- **Backend:** API REST desarrollada con FastAPI y SQLAlchemy.
- **Base de datos:** persistencia en PostgreSQL.

Esta separación facilita el mantenimiento y permite que cada componente tenga responsabilidades claras. El frontend consume la API mediante rutas bajo `/api`; el backend procesa la lógica de negocio y la base de datos conserva la información persistente.

4.4. Proxy Inverso

nginx se utilizó como servidor web y proxy inverso. Su función es servir los archivos estáticos generados por la compilación de React y redirigir las solicitudes de API hacia Uvicorn, que ejecuta la aplicación FastAPI en el puerto 8000.

Esta configuración permite exponer un único punto de entrada HTTP, manteniendo oculto el puerto interno del backend y simplificando el acceso desde el navegador.

5. Desarrollo del Proyecto

5.1. Planificación

El desarrollo se organizó en fases: base de datos, backend, frontend, nginx, publicación del informe y acceso externo. Esta división permitió validar cada capa de manera independiente antes de integrarlas.

La consigna establecía dos contenedores: uno para el blog personal y otro para la base de datos. La implementación final respetó esa separación y agregó funciones administrativas para facilitar la actualización del sitio sin modificar archivos manualmente.

5.2. Infraestructura Implementada

Contenedor	Nombre	IP	Servicios
Aplicación	43362480A	172.16.90.215	nginx, frontend React, backend FastAPI y Uvicorn
Base de datos	43362480DB	172.16.90.207	PostgreSQL 16

Tabla 1: Contenedores utilizados en la implementación

El contenedor de aplicación concentra los servicios necesarios para atender solicitudes web. El contenedor de base de datos queda separado para mantener la persistencia aislada y permitir que el backend se conecte mediante la red interna.

5.3. Pila Tecnológica

Componente	Tecnología
Frontend	React 19, TypeScript, Vite y Tailwind CSS
Backend	Python, FastAPI, SQLAlchemy y Uvicorn
Base de datos	PostgreSQL 16
Servidor web	nginx
Infraestructura	Proxmox VE con contenedores LXC
Autenticación	JWT, OAuth2 con tokens Bearer y bcrypt

Tabla 2: Tecnologías utilizadas

5.4. Backend

El backend fue desarrollado con FastAPI. Al iniciar, crea las tablas necesarias mediante SQLAlchemy y registra un perfil inicial con el nombre del alumno si no existe información previa.

La configuración de conexión a PostgreSQL se toma desde variables de entorno:

```
DB_USER
DB_PASSWORD
DB_HOST
DB_PORT
DB_NAME
```

Los módulos principales del backend son:

- `main.py`: inicializa la aplicación FastAPI, CORS, routers y login.

- `database.py`: define la conexión a PostgreSQL y la sesión de SQLAlchemy.
- `models.py`: declara los modelos `Post` y `Profile`.
- `schemas.py`: define los esquemas de entrada y salida con Pydantic.
- `auth.py`: gestiona autenticación, hash de contraseña y emisión de tokens JWT.

5.5. API

La API expone puntos de acceso públicos para lectura y puntos de acceso protegidos para administración. Las operaciones de creación, edición y eliminación requieren un token válido.

Método	Ruta	Descripción
POST	/api/auth/login	Autentica al administrador y devuelve un token JWT.
GET	/api/posts	Lista las entradas publicadas.
GET	/api/posts/{id}	Obtiene el detalle de una entrada.
POST	/api/posts	Crea una nueva entrada. Requiere autenticación.
PUT	/api/posts/{id}	Actualiza una entrada existente. Requiere autenticación.
DELETE	/api/posts/{id}	Elimina una entrada. Requiere autenticación.
GET	/api/profile	Devuelve los datos del perfil.
PUT	/api/profile	Actualiza nombre y biografía. Requiere autenticación.
POST	/api/profile/photo	Carga la foto de perfil. Requiere autenticación.
GET	/api/system	Devuelve información del contenedor de aplicación.
GET	/api/system/stream	Envía métricas en vivo mediante Server-Sent Events.
GET	/api/system/schema	Devuelve el esquema de la base de datos para visualizarlo en el frontend.
GET	/api/system/health	Verifica el estado de la API y la conexión a la base de datos.
GET	/api/system/pdf-status	Indica si el informe PDF está publicado y su tamaño.
POST	/api/system/upload-pdf	Carga el informe en PDF. Requiere autenticación.
DELETE	/api/profile/photo	Elimina la foto de perfil y vuelve a la imagen de GitHub. Requiere autenticación.

Tabla 3: Puntos de acceso principales del backend

5.6. Modelo de Datos

La base de datos utiliza dos entidades principales:

Tabla	Propósito
posts	Almacena las entradas del blog, contenido, etiquetas y fechas de creación y actualización.
profile	Almacena nombre, biografía y foto del alumno.

Tabla 4: Tablas principales de PostgreSQL

5.7. Frontend

El frontend fue construido con React, TypeScript y Vite. La aplicación utiliza React Router para definir las rutas principales:

- `/`: página principal con listado de posts, búsqueda, etiquetas y datos resumidos del perfil.
- `/post/:id`: detalle de una publicación con soporte para Markdown, tablas y bloques de código.
- `/about`: información del alumno, descripción del proyecto, acceso al informe PDF, stack técnico, diagrama de infraestructura y métricas del sistema.
- `/login`: inicio de sesión del administrador.
- `/admin`: panel privado para administrar publicaciones, perfil y registro de cambios.

El diseño visual adopta una estética sobria de terminal, con colores oscuros, tipografía compacta y navegación simple. Para mejorar la experiencia de lectura, las publicaciones soportan Markdown con resaltado de sintaxis y botón para copiar fragmentos de código.

5.8. Administración del Blog

El panel de administración permite actualizar el contenido sin acceder por consola al servidor. Sus funciones principales son:

- Crear, editar y eliminar publicaciones.
- Agregar etiquetas a cada post.
- Modificar nombre y biografía del perfil.
- Cargar una foto de perfil, con recorte cuadrado antes de subirla.
- Subir el informe en PDF para publicarlo en la sección `/about`.

La autenticación se realiza con usuario y contraseña configurables mediante variables de entorno. Una vez autenticado, el frontend guarda el token en el navegador y lo envía en el encabezado `Authorization` para las operaciones protegidas.

5.9. Monitoreo e Información Técnica

Además de las funciones requeridas por la consigna, se agregó una sección técnica en la página /about. Esta sección muestra:

- Arquitectura de la infraestructura con los contenedores y servicios.
- Estado de la API y de la base de datos.
- Información del sistema operativo, hostname, IP, CPU, memoria, disco y uptime.
- Diagrama de entidad-relación generado a partir del esquema real de PostgreSQL.

Para las métricas en vivo se utiliza el punto de acceso /api/system/stream, implementado con Server-Sent Events. El backend recopila datos con `psutil` y los envía al navegador cada segundo.

5.10. Configuración de nginx

nginx sirve la compilación estática de React desde /var/www/blog/dist. Las rutas del frontend se resuelven con `try_files`, de modo que React Router pueda manejar rutas internas como /about o /admin.

Las solicitudes a /api/ se reenvían a FastAPI:

```
location /api/ {  
    proxy_pass http://127.0.0.1:8000;  
}
```

Para el punto de acceso de métricas en vivo se deshabilitó el almacenamiento intermedio de nginx, evitando que los eventos SSE queden retenidos antes de llegar al navegador.

5.11. Publicación del Informe

La consigna solicita que el informe en PDF esté disponible desde el blog. Para cumplirlo, la página /about incluye un visor y un enlace de descarga. El archivo se sube desde el panel de administración y se almacena en el servidor bajo:

```
/var/www/blog/dist/static/informe.pdf
```

nginx lo sirve bajo la ruta /static/informe.pdf (o /43362480/static/informe.pdf con el prefijo del proxy externo). El directorio static/ se excluye del proceso de sincronización durante los deploys para preservar el archivo entre actualizaciones.

5.12. Publicación bajo un Prefijo de URL

El acceso externo al blog se realiza a través del dominio `nap.frt.utn.edu.ar`, donde un proxy inverso reenvía las solicitudes al contenedor de aplicación bajo el prefijo /43362480/. Esto requirió configurar el enrutador de React con el basenname correspondiente y prefijar todas las llamadas a la API con ese valor.

El prefijo se inyecta en tiempo de compilación mediante la variable de entorno `VITE_BASENAME=/43362480` definida en el archivo `.env.production`. En el código, todas las rutas absolutas se construyen a partir de esta variable.

Para resolver un problema adicional, el proxy externo devolvía 501 Not Implemented para archivos .js y .css. La solución fue utilizar `vite-plugin-singlefile`, un plugin de Vite que inlinea todo el JavaScript, CSS y los recursos SVG directamente dentro del archivo `index.html`. El resultado es un único archivo HTML de aproximadamente 3,5 MB que no requiere que el proxy sirva assets adicionales.

6. Pruebas Realizadas

Las pruebas se enfocaron en validar el funcionamiento de cada capa y la integración entre ellas:

- Acceso a la página principal y carga de publicaciones desde la API.
- Visualización del detalle de posts en Markdown.
- Inicio de sesión del administrador.
- Creación, edición y eliminación de publicaciones.
- Actualización del perfil y carga de foto.
- Conexión del backend con PostgreSQL.
- Respuesta del punto de acceso de verificación de estado.
- Visualización de métricas del sistema en tiempo real.
- Enrutamiento de nginx para frontend y API.

Estas pruebas permiten comprobar que el servicio funciona como una aplicación web integrada y no como componentes aislados.

7. Resultado Final

El resultado final es un blog personal desplegable sobre Proxmox, con separación entre aplicación y base de datos. La aplicación permite publicar contenido, administrar datos personales, mostrar información de infraestructura y descargar el informe del trabajo práctico.

La arquitectura implementada cumple con los requerimientos principales de la consigna:

- Datos personales del alumno.
- Imagen personal configurable.
- Página principal actualizable mediante posts.
- Informe final disponible en PDF desde el blog.
- Uso de dos contenedores LXC en Proxmox.
- Base de datos PostgreSQL separada de la aplicación.
- Exposición web mediante nginx.

8. Conclusión

El trabajo permitió integrar los conceptos de virtualización con el desarrollo y despliegue de una aplicación web real. La separación en contenedores facilitó aislar responsabilidades: un contenedor dedicado a la aplicación y otro a la persistencia.

La utilización de nginx como proxy inverso permitió publicar el frontend y el backend bajo un mismo punto de entrada. FastAPI y PostgreSQL resolvieron la lógica de negocio y la persistencia, mientras que React aportó una interfaz de usuario dinámica para administrar el contenido.

Como mejora adicional, se incorporaron métricas del sistema, visualización de infraestructura y administración del perfil, lo que convierte al blog en una entrega más completa y demostrable durante la defensa presencial.